系统架构设计师

案例分析 | 真题精选练习

必选题 + 四选二

数据库 / MySQL方向 | Redis | Cache-Aside | Web

跨年份公开回忆版组卷 01 | recalled_real | 题目与答案

姓名：_____	练习日期：_____
开始时间：_____	结束时间：_____
选做题号：_____	实际用时：_____分钟

作答规则

试题一必做；试题二至试题五任选两道。每道25分，共75分。练习建议限时90分钟，合格线45分。

本卷面向数据库、缓存和Web方向跨年份编排，四道选做题均为软件方向；并非某一年的完整原卷。原考试未必同时提供四道这类题。

保留年份、原题号、题干、小问、分值及填空形式。来源为公开回忆或整理版本，不称为官方原卷。部分文字作排版和明显错字修正。

每道大题后直接附参考答案，按小问给出可书写的答案和采分关键词。答案不是官方评分细则，技术争议处单独说明。

试题一的必选身份沿用2024下来源；试题二至五在本练习中均为选做题。不要把必选题范围固定理解为只考ATAM。

本卷题目

练习题号	年份 / 原题号	方向
一（必选）	2024下 / 试题一	质量属性与故障检测
二（选做）	2026上 / 试题三	数据库并发控制与Redis
三（选做）	2025下 / 批次一试题四	Redis类型、持久化、淘汰
四（选做）	2024下 / 试题二	Cache-Aside与缓存一致性
五（选做）	2024下 / 试题四	Web架构、分词器、RESTful

试题一 必选题

质量属性与故障检测

2024年下半年 | 原试题一 | 25分 | 公开回忆整理版

【说明】

某公司拟构建一个基于大语言模型的智能平台，该框架将依据多个大语言模型来协作处理用户提交的任务请求，任务求解过程的状态数据将被记录在任务板中，各个大语言模型根据任务板上任务的实时求解状态，来确定它们当前是否需要被调用以进一步处理该任务，并在处理结束之后将任务的最新状态更新到任务板上。 基于该架构的开发任务，公司召开项目讨论会。会上，项目组介绍了环境需求，主要包括：

（a）系统可支持用户在任务板上查看任务的状态和结果，并允许用户多次调用大型语言模型以进一步优化处理结果。

（b）系统可支持任务数据的导入导出，数据导入导出在1分钟内结束

（c）在数据服务器发生故障时，系统应能够立刻切换到备份服务器，并保证数据同步，以确保系统的不间断的服务。

（d）在环境正常负荷情况下，用户在任务板上查询任务状态和结果的响应时间应在2秒内。

（e）系统应确保用户数据和操作记录的安全，防止未经授权的访问。

（f）系统要求支持多语言接口，并提供查询词自动补全和搜索关联功能。

（g）系统应支持扩容，以容纳更多的用户和任务。扩容需求的搭建应在两名运维人员工作的情况下在5天内完成。

（h）系统部署在云服务器上。当云服务器出现故障时，系统应在1分钟内检测出故障，并在1小时之内恢复。

（i）系统支持根据用户任务类型调用相应的大语言模型对任务进行处理。

（j）用户可以在任务板上搜索历史任务和结果，搜索结果应在3秒内返回。

试题一｜问题与填空表（续）

【问题1】（14分）

为了辅助架构师张工完成系统架构设计，首先需对上述需求进行分析。请分析需求（a）~（j），补充完善下表中（1）~（7）的空白处。

需求	功能性需求 / 具体质量属性需求名称
(a)	功能性
(b)	（1）_____
(c)	可靠性
(d)	（2）_____
(e)	（3）_____
(f)	（4）_____
(g)	（5）_____
(h)	可用性
(i)	（6）_____
(j)	（7）_____

表格按希赛网公开回忆版图表转录；来源见末页。

【问题2】（11分）

针对可用性需求（h），张工提出可采用ping/echo策略完成故障检测，但李工认为从系统资源利用率的角度出发，采用心跳策略完成故障检测更优。

（1）请分别说明如何采用ping/echo策略和心跳策略来完成可用性的故障检测。

（2）请解释李工认为心跳策略更优的原因。

试题一｜参考答案与采分关键词

参考作答由回忆版答案结合技术文档校正，不是官方标准答案或官方评分细则。

【问题1】（14分）需求分类填空

空号	答案	对应需求与判断依据
（1）	性能	（b）导入导出在1分钟内完成
（2）	性能	（d）查询响应时间在2秒内
（3）	安全性	（e）防止未经授权访问
（4）	功能性	（f）多语言接口、自动补全和关联搜索
（5）	可修改性	（g）两名运维人员在5天内完成扩容
（6）	功能性	（i）按任务类型调用相应模型
（7）	性能	（j）历史搜索在3秒内返回

判断时优先看约束：响应时限对应性能；未授权访问对应安全性；修改或扩容所需人力与时间对应可修改性；“能做什么”对应功能性。表中（c）可靠性、（h）可用性已给出，不另填空。

【问题2】（11分）Ping/Echo与心跳

（1）Ping/Echo：监控组件周期性向被监控云服务器发出Ping请求，服务器正常时返回Echo；超时或连续多次未收到响应，则判断服务器可能故障，并触发告警或恢复。

心跳：被监控云服务器周期性主动向监控组件发送心跳消息；监控组件记录最近一次接收时间，超时或连续缺失心跳时判断可能故障。

（2）心跳通常使用简短的单向消息，减少主动探测的请求与响应往返、监控端的请求调度和等待，以及网络与处理开销，因此在大规模监控场景下资源利用率较好。应配置发送周期和超时阈值，满足1分钟内故障检测，并由恢复机制满足1小时内恢复。

采分关键词：主动方、周期发送、超时判故障、单向简短消息、减少往返与开销。不要写“心跳必然长连接、Ping必然短连接”或“未收到一次就确定故障”。

试题二 选做题（四选二）

数据库并发控制与Redis优化

2026年上半年 | 原试题三 | 25分 | 公开回忆版

【说明】

某电商公司在商品秒杀场景中，大量的用户会对秒杀中的商品进行访问和购买操作。如果秒杀系统缺少并发控制机制，将会导致商品超卖或者数据不一致等问题。

【问题1】（8分）

张工建议采用悲观锁进行并发访问控制，请按锁的行为分类，列举出至少两种不同类别的悲观锁，并用200字以内的文字分析出它们的差别。

【问题2】（11分）

李工建议采用乐观锁进行并发访问控制，请用300字以内的文字从应用和DBMS管理层面给出乐观锁的实现方案，并分析采用悲观锁与乐观锁的优缺点。

【问题3】（6分）

结合Redis缓存列举至少3个高并发控制场景优化策略。

试题二｜参考答案与采分关键词

参考作答由回忆版答案结合技术文档校正，不是官方标准答案或官方评分细则。

【问题1】（8分）共享锁与排他锁

共享锁（S锁、读锁）用于保护读取，同一数据可由多个事务同时持有共享锁，但与其他事务的排他锁冲突。排他锁（X锁、写锁）用于保护修改，同一数据同时只允许一个事务持有排他锁，与其他事务的共享锁和排他锁均冲突。共享锁适于并发读取，排他锁保证更新的互斥性。

采分关键词：S/S兼容、S/X冲突、X/X冲突、读与写。MySQL InnoDB的普通SELECT可通过MVCC读历史版本，因此不要把排他锁说成“阻止所有普通查询”。

【问题2】（11分）乐观锁实现与优缺点

应用层：增加版本号字段，读取数据与版本；更新时使用旧版本作为条件，并同时递增版本。影响行数为0表示并发冲突，应限次重试或返回失败。DBMS层：采用乐观并发控制，事务读取快照并记录读写集，提交前验证是否与并发事务冲突；通过则提交，否则回滚重试。悲观锁适合冲突多的更新，但会阻塞，可能死锁；乐观锁适合冲突少、读多写少，减少提前加锁，但高冲突下反复重试成本大。

实现示例（辅助理解）

UPDATE stock SET quantity=quantity-1, version=version+1 WHERE product_id=? AND version=? AND quantity>0;

采分关键词：版本号、条件更新、冲突检测、回滚重试、适用场景。上述DBMS描述是通用乐观并发控制；MySQL的MVCC不等于更新完全不加锁，InnoDB写操作仍会使用锁。

【问题3】（6分）至少三个Redis优化场景

① 秒杀扣库存：库存预存Redis，用Lua脚本将“检查库存、检查购买资格、扣减、登记”原子执行。成功请求通过可靠队列异步落库，并用订单唯一键保障幂等、定期对账。

② 并发修改：WATCH监视目标键，读取并校验后用MULTI/EXEC提交；键发生变化则事务中止，采用有限重试。

③ 共享资源互斥：SET lock:key token NX PX timeout原子加锁，释放时用Lua校验token并删除；合理设置有效期，避免误删其他持有者的锁。

④ 热点读请求：缓存商品详情；热点失效时通过互斥重建或逻辑过期加异步刷新减少并发回源，再配合限流保护数据库。

本问写出三个“场景＋策略”即可。不要只写DECR就声称不会超卖；不能用不存在的SETNXEX命令。

试题三 选做题（四选二）

Redis数据类型、持久化与内存淘汰

2025年下半年 | 批次一原试题四 | 25分 | 公开整理版

【说明】

某互联网文化发展公司因业务发展，需要建立网上社区平台，为用户提供一个对网络文化产品（如互联网小说、电影、漫画等）进行评论、交流的平台。该平台的部分功能如下：

(a) 用户帖子的评论计数器；

(b) 支持粉丝列表功能；

(c) 支持标签管理；

(d) 支持共同好友功能等；

(e) 提供排名功能，如当天最热前10名帖子排名、热搜榜前5排名等；

(f) 用户信息的结构化存储；

(g) 提供好友信息的发布/订阅功能。

该系统在性能上需要考虑高性能、高并发，以支持大量用户的同时访问。开发团队经过综合考虑，在数据管理上决定采用Redis+数据库（缓存+数据库）的解决方案。

【问题1】（10分）

Redis支持丰富的数据类型，并能够提供一些常见功能需求的解决方案。请选择题干描述的(a)~(g)功能选项，填入表中(1)~(5)的空白处。

数据类型	存储的值	可实现的业务功能
STRING	字符串、整数或浮点数	（1）_____
LIST	列表	（2）_____
SET	无序集合	（3）_____
HASH	包括键值对的无序散列表	（4）_____
ZSET	有序集合	（5）_____

表格按软考在线公开题库样张第34页转录；按题干功能字母填写，可填多个。

【问题2】（7分）

Redis有两种持久化方式：RDB和AOF。请从磁盘更新频率、数据安全、数据一致性、重启性能和数据文件大小五个方面比较两种方式，并简要说明开发团队选择RDB的原因。

【问题3】（8分）

缓存中存储当前的热点数据，Redis为每个KEY值都设置了过期时间。为了清除非热点数据，Redis选择"定期删除+惰性删除"策略。如果该策略失效，Redis内存使用率会越来越高。请简要描述该策略的失效场景，并给出三种内存淘汰机制。

试题三｜参考答案与采分关键词

参考作答由回忆版答案结合技术文档校正，不是官方标准答案或官方评分细则。

【问题1】（10分）数据类型与功能

空号 / 类型	题库口径填空	理由
（1）STRING	（a）	INCR/DECR实现评论计数
（2）LIST	（b）、（g）	列表；（g）归类存在回忆版本差异，见下
（3）SET	（c）、（d）	标签去重；集合交集求共同好友
（4）HASH	（f）	以字段保存结构化用户信息
（5）ZSET	（e）	按score排序生成榜单

口径说明：公开题库样张把（g）归入LIST；本题回忆文字为“好友信息发布/订阅”，工程上真正的发布订阅应使用PUBLISH/SUBSCRIBE，不能说LIST原生提供Pub/Sub。记表格答案时同时保留这一差异。

【问题2】（7分）RDB与AOF

维度	RDB	AOF
磁盘更新频率	周期或条件触发生成快照	持续追加写命令；刷盘按策略
数据安全	可能丢失最近快照之后的数据	通常损失窗口更小；everysec通常约1秒
恢复的一致性 / 新鲜度	恢复快照时刻的数据，之后的写入可能丢失	重放已持久化操作，通常恢复得更接近故障前
重启性能	通常较快，直接加载快照	纯命令重放通常更慢；混合持久化有所不同
文件大小	快照通常紧凑	日志通常较大，可通过重写压缩

选择RDB的理由：本系统重视宕机后的快速重启，RDB通常加载较快、文件较紧凑；作为数据库前的缓存，可从数据库重建。是否接受快照间隔内的丢失，仍需以业务要求为准。

采分关键词：快照、追加日志、刷盘策略、恢复速度、损失窗口、日志重写。持久化不直接等于主从一致性；AOF也不是无条件零丢失。

【问题3】（8分）过期删除与内存淘汰

定期删除有抽样与执行时间预算，部分过期键不能立即被扫描；惰性删除需访问键才触发。若这些键长期不被访问，就可能暂时占用内存。另外，无过期时间的键或不断增长的数据也会造成内存压力。达到maxmemory限制时，由配置的淘汰策略处理。

① allkeys-lru：从所有键中淘汰近似最近最少使用的键。② volatile-lru：仅从设置了过期时间的键中淘汰近似最近最少使用的键。③ allkeys-random：从所有键中随机淘汰。

采分关键词：抽样、未访问、过期删除滞后、maxmemory、三个策略及范围。区分“按TTL过期”与“内存达到上限后的淘汰”。

试题四 选做题（四选二）

Cache-Aside缓存一致性

2024年下半年 | 原试题二 | 25分 | 公开回忆整理版

【说明】

为了提升某购物网站的用户使用体验，采用数据库缓存技术将经常访问的商品信息存储在一个临时高速数据存储层中，这个临时存储层使得未来对这些内容的请求响应比通过访问主数据库更快。为了确保快速、及时获取最新数据，项目组决定采用缓存数据技术开展项目设计，经讨论，决定采用Cache-Aside策略作为缓存手段。

【问题1】（10分）

启用Cache-Aside缓存策略，当用户查询商品信息时，此请求会从应用程序服务器发送请求到数据库服务器，依据缓存环境处理，然后返回所请求的信息，请补充完善图1中（1）~（5）处的内容，协助设计师达成缓存框架中的数据读取模块的设计方案。

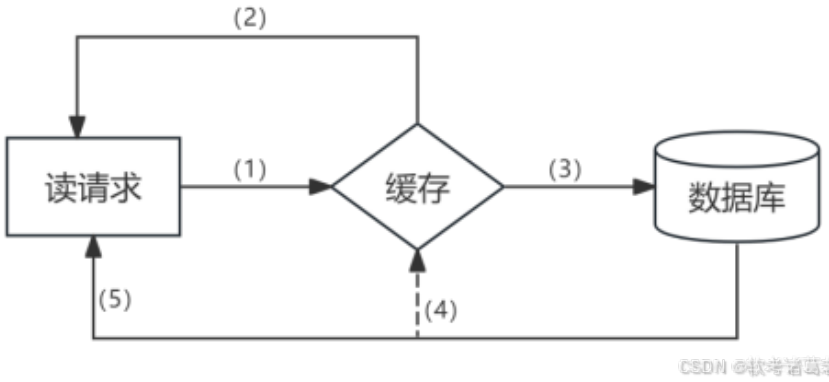


图1 数据读取流程（公开回忆版图）

【问题2】（6分）

使用Cache-Aside缓存策略，当用户更新商品信息时，此请求会从应用服务器发至数据库服务器，并进行缓存处理，请补充完善图2中（1）和（2）处的内容，协助工程师完成缓存系统中的信息更新模块的设计方案。



图2 数据更新流程（公开回忆版图）

【问题3】（9分）

李工设计了多线程并发数据访问方案，采用线程1执行数据更新，线程2执行数据读取。王工认为这样的设计方案会存在数据不一致现象，请说明李工所设计的方案为什么会出现数据库与缓存中的数据不一致的情况，并给出3种或以上的解决方案。

试题四｜参考答案与采分关键词

参考作答由回忆版答案结合技术文档校正，不是官方标准答案或官方评分细则。

【问题1】（10分）读取流程五个空

（1）向缓存请求读取目标商品信息。

（2）缓存命中，直接返回商品信息。

（3）缓存未命中，向数据库查询商品信息。

（4）将数据库查询结果写入缓存。

（5）将查询到的商品信息返回给调用方。

【问题2】（6分）更新流程两个空

（1）更新数据库中的目标商品信息。

（2）删除该商品对应的缓存。

采分关键词：先更新数据库，后删除缓存。采用本卷仓库版本的Cache-Aside更新口径；不要直接把旧回忆解析中“写入缓存”当成唯一答案。

【问题3】（9分）不一致原因与解决方案

原因：数据库与缓存是两个独立系统，更新数据库、删除缓存，以及读库后回填缓存无法作为一个普通数据库事务原子完成。并发读写交错或缓存删除失败，都可能导致缓存保存旧值。

典型交错：缓存未命中，读线程查到旧值v1；写线程把数据库更新为v2并删除缓存；读线程随后把v1回填缓存。此时数据库是v2，缓存是v1。

方案① 可靠失效：先更新数据库，再删缓存；删除失败时通过可靠消息和重试补偿，并保持消费幂等。

方案② Binlog/CDC：订阅数据库提交后的变更日志，驱动缓存失效；记录消费位置、重试并对账，使漏删可以追补。

方案③ 并发协调：针对同一业务键，使读库回填与写库失效采用共同的互斥或串行机制；也可使用版本号并原子比较，防止旧版本覆盖新版本。

补充：配置合理TTL限定陈旧窗口；延迟双删可降低特定读写交错的风险，但依赖延迟覆盖回填时序，并需删除失败重试。

采分关键词：非原子、交错执行、旧值回填、失效失败、至少三种方案。消息、CDC、TTL等一般实现最终一致；不能只写“设TTL”就声称强一致。

试题五 选做题（四选二）

Web架构、Elasticsearch分词器与RESTful

2024年下半年 | 原试题四 | 25分 | 公开回忆版

【说明】

基于Elasticsearch分词的商品推荐系统，使用微信小程序接入。

【问题1】（6分）

说明Standard、Simple、Whitespace、Keyword分词器各自的分词原理和特点差异。

【问题2】（12分）

根据商品推荐系统架构图，从下列名称中选择合适内容，填写图中（1）~（8）的空白处。架构图见下一页。

分层候选：接入层、显示层、网络层、应用层、业务逻辑层、控制层、数据层。

技术候选：MyBatis、Nginx、Flink、JavaScript、Node.js、RESTful、Elasticsearch、Kafka。

【问题3】（7分）

RESTful架构有什么特点？它如何实现前后端分离？

本题背景及问法取自仓库回忆版，填空图取自公开回忆复原图；采用其已有空号，不附已填答案图。

试题五 | 问题2架构图

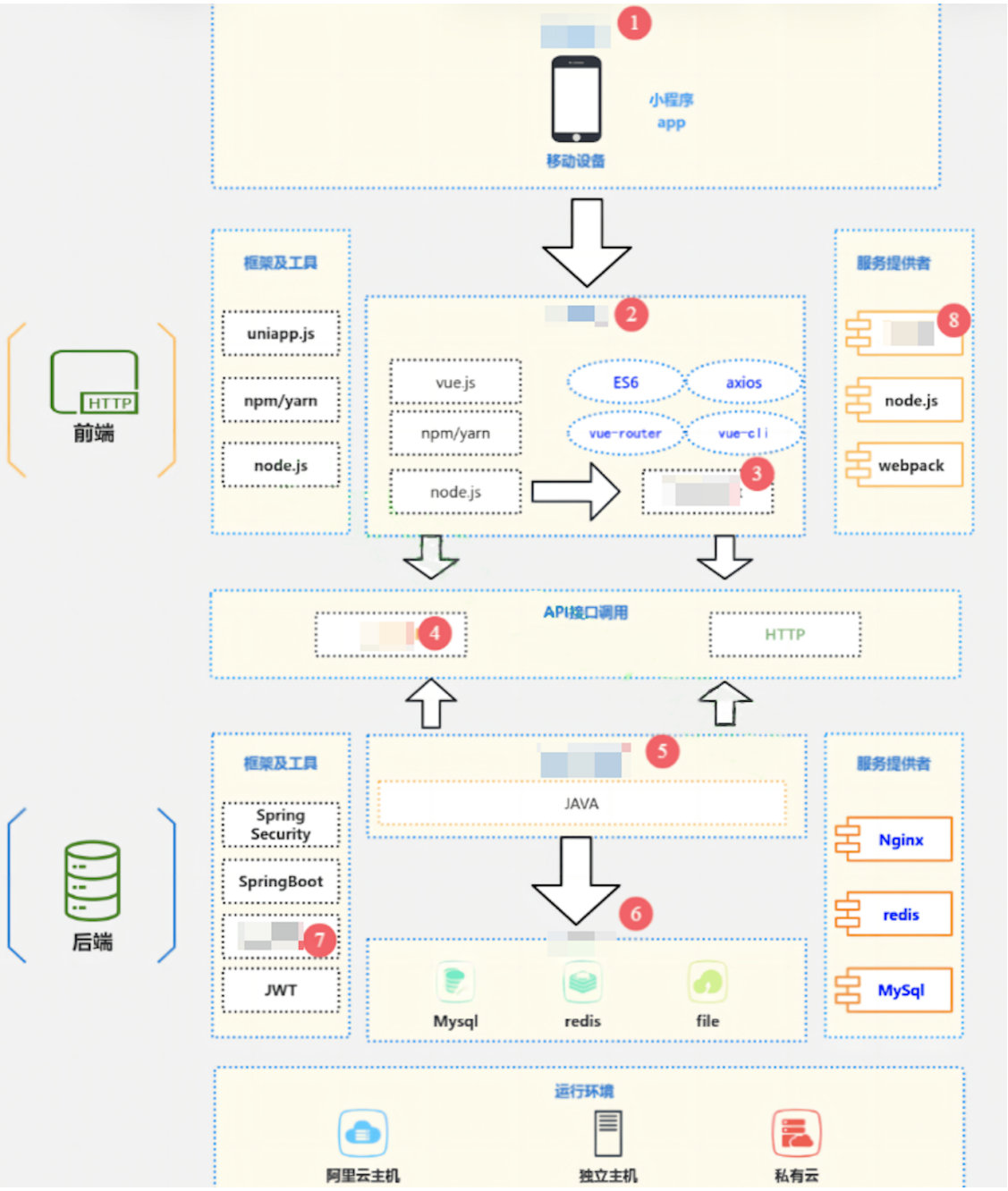


图3 商品推荐系统架构（公开回忆复原图，非官方原图）

空号	填写内容	空号	填写内容
(1)	_____	(5)	_____
(2)	_____	(6)	_____
(3)	_____	(7)	_____
(4)	_____	(8)	_____

试题五｜参考答案与采分关键词

参考作答由回忆版答案结合技术文档校正，不是官方标准答案或官方评分细则。

【问题1】（6分）四种分词器

分词器	可直接书写的答案
Standard	默认分词器，按Unicode词边界切分，并进行小写化；通常去除标点边界，保留数字词项。中文不等同于完整的语义词典分词。
Simple	以非字母字符分割，丢弃非字母字符，并把字母转为小写。
Whitespace	只按空白字符切分；保留大小写及词内标点、数字。
Keyword	不切分，整段输入作为一个词项，默认不改大小写。

示例：输入Hello, WORLD! 2026。Standard得到hello / world / 2026；Simple得到hello / world；Whitespace得到Hello, / WORLD! / 2026；Keyword保留整串。

【问题2】（12分）架构图八个空

空号	答案	空号	答案
（1）	显示层	（5）	业务逻辑层
（2）	控制层	（6）	数据层
（3）	JavaScript	（7）	MyBatis
（4）	RESTful	（8）	Nginx

填空按本卷公开复原图的空号匹配；图中的层名采用来源口径，不把它当成所有Web系统的标准分层名称。

【问题3】（7分）RESTful与前后端分离

REST以资源为核心，用URI标识资源，通过统一接口操作资源，具有客户/服务器、无状态、可缓存、分层等约束。HTTP API通常用GET、POST、PUT、DELETE等方法表达操作语义，用状态码表达结果，传输JSON等资源表示。

前端负责页面与交互，后端负责业务与数据。双方约定API路径、请求参数、响应结构、认证和错误格式，通过HTTP接口交换数据，从而可独立开发与部署。无状态要求每次请求携带处理所需的上下文；不代表服务器不能保存业务数据。

采分关键词：资源、URI、统一接口、HTTP方法、无状态、接口契约、职责分离、独立部署。

来源与练习记录

来源说明

本册使用仓库现有回忆题面，并补回公开来源的填空图表。商业题库网站不等同于考试主管部门；本册不采用其“官方题库”称谓。不同回忆版本的措辞可能有差异。

题一题面、题四题面、题五题面

past-papers/case-by-year/2024下.md

题二题面

past-papers/case-by-year/2026上.md；source-pdfs/2026上/回忆版PDF

题三题面

past-papers/case-by-year/2025下.md（批次一）

题一需求表

https://www.educity.cn/rk/5437194.html

题三Redis数据类型表

https://www.rkpass.cn/yangzhang/4_80_1.pdf（第34页）

题四流程图

https://jishuzhan.net/article/1900600614273495041

题五填空图

https://jishuzhan.net/article/2041681427436208129

参考答案｜技术校核来源

以下官方技术文档用于校正锁、缓存和分词器等实现细节。题目分值沿用回忆题面，答案不另造逐条官方分数。

悲观锁兼容关系

https://dev.mysql.com/doc/refman/8.4/en/innodb-locking.html

MySQL一致性非锁定读

https://dev.mysql.com/doc/refman/8.4/en/innodb-consistent-read.html

Redis WATCH事务

https://redis.io/docs/latest/develop/using-commands/transactions/

Redis分布式锁

https://redis.io/docs/latest/develop/clients/patterns/distributed-locks/

Redis持久化

https://redis.io/docs/latest/management/persistence/

Redis内存淘汰

https://redis.io/docs/latest/develop/reference/eviction/

Redis发布订阅

https://redis.io/docs/latest/develop/pubsub/

Elasticsearch分词器

https://www.elastic.co/docs/reference/text-analysis/analyzer-reference

记录区

项目	填写
必选题得分	_____ / 25
选做题____得分	_____ / 25
选做题____得分	_____ / 25
合计	_____ / 75
最需要补的题号 / 小问	_____

生成或阅读本册不构成掌握证据。学习成绩应以实际独立作答及评分为准。